

Distributed Real-time systems and Delays

Henrik Schiøler, Aalborg University

System (Model/Aspect) Types

- Systems of Systems (distributed objectives and decisions)
- Cyber-Physical Systems (communication, computation, control)
- Internet Of Things (embedded devices connected by internet)
- Sensor-Actuator Networks (sensors and actutors wirelessly connected)

All distributed systems with performance requirements:

- timing
- reliability

What is a Real-Time System?

- ☞ A real-time system is a computer system in which the correctness of the system behavior depends not only on the logical results of the computations but also on the time when the results are produced.
- ☞ Real-time systems usually are in strong interaction with their physical environment. They receive data, process it, and return results *in right time*.

Examples:

- Process control systems
- Computer-integrated manufacturing systems
- Aerospace and avionics systems
- Automotive electronics
- Medical equipment
- Nuclear power plant control
- Defence systems
- Consumer electronics
- Multimedia
- Telecommunications

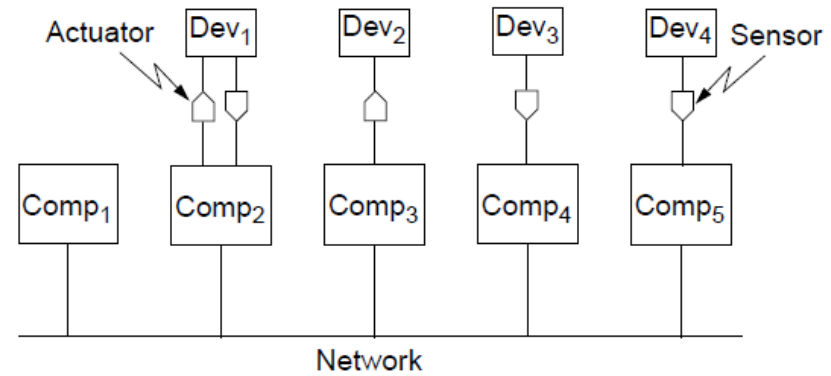


Distributed Real-Time Systems

Real-time systems very often are implemented as distributed systems. Some reasons:

- Fault tolerance
- Certain processing of data has to be performed at the location of the sensors and actuators.
- Performance issues.

Distributed Real-Time Systems (cont'd)





Real -Time Systems

Some Typical Features

- They are time - critical.
The failure to meet time constraints can lead to degradation of the service or to catastrophe.
- They are made up of concurrent processes (tasks).
- The processes share resources (e.g. processor) and communicate to each other.
This makes scheduling of processes a central problem.
- Reliability and fault tolerance are essential.
Many applications are safety critical.
- Very often, such systems are dedicated to a specific functionality.
They perform a certain specific job which is fixed.
This is opposed to standard, general purpose computing, where several different programs are run on a computer to provide different services.
- Such systems are very often embedded in a larger system, like a car, CD-player, phone, camera, etc.



Soft and Hard Deadlines

- ⇒ Time constraints are often expressed as deadlines at which processes have to complete their execution.

A deadline imposed on a process can be:

- Hard deadline: has to be met strictly, if not ⇒ "catastrophe".
- should be guaranteed a-priori, off-line.
- Soft deadlines: processes can be finished after its deadline, although the value provided by completion may degrade with time.
- Firm deadlines: similar to hard deadlines, but if the deadline is missed there is no catastrophe, only the result produced is of no use any more.



Predictability

- ☞ Predictability is one of the most important properties of any real-time system.
- ☞ Predictability means that it is possible to guarantee that deadlines are met as imposed by requirements:
 - hard deadlines are always fulfilled.
 - soft deadlines are fulfilled to a degree which is sufficient for the imposed quality of service.

Some problems concerning predictability:

- Determine worst case execution times for each process.
- Determine worst case communication delays on the interconnection network.
- Determine bound on clock *drift* and *skew* (see later).
- Determine time overheads due to operating system (interrupt handling, process management, context switch, etc.).
- After all the problems above have been solved, comes the "big question":
Can the given processes and their related

Causes of delay

- Propagation

Speed of light – around the globe in around 0.12 sec

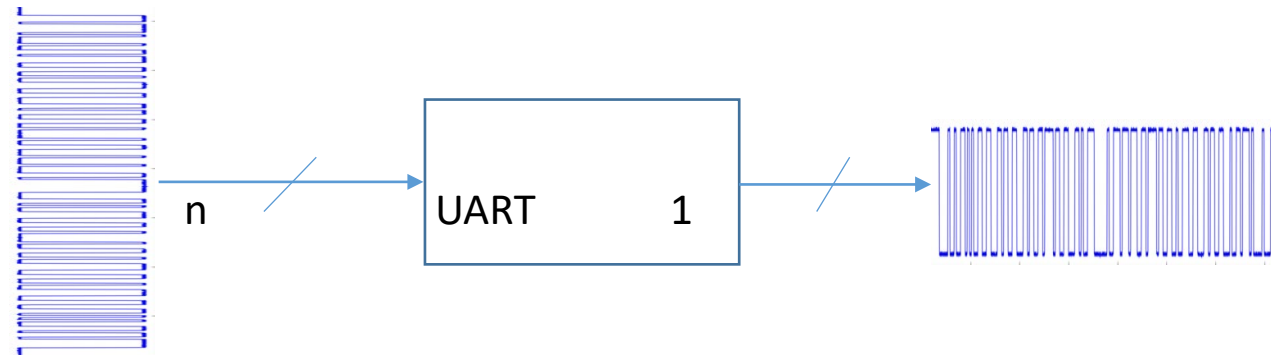
Through DK in around 1mS

Typically constant

Often considered insignificant

Causes of delay

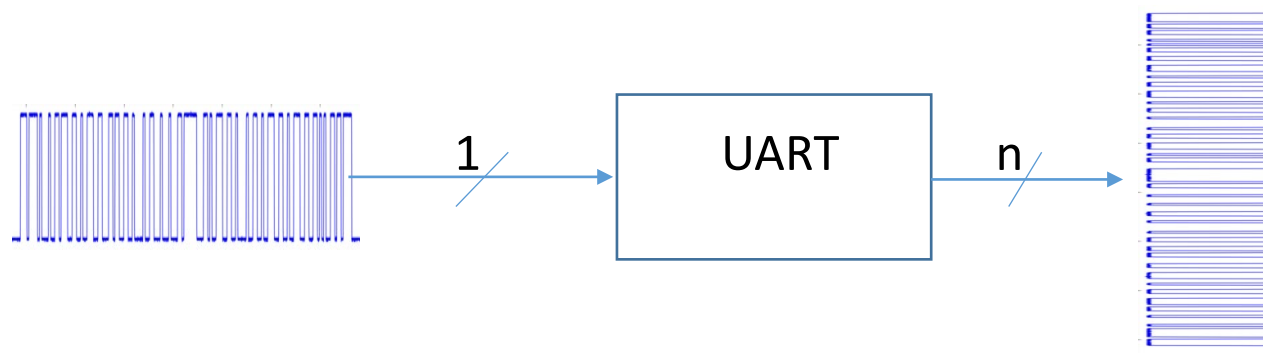
- Serialization



- Delay depends on serialization speed (bpS, Mbps)
- Serialization speed proportional to frequency consumption
- Complementary to reliability
- Complementary to range

Causes of delay

- Packetizing



- Delay depends on serialization speed (bpS, Mbps)

Causes of delay

- Processing

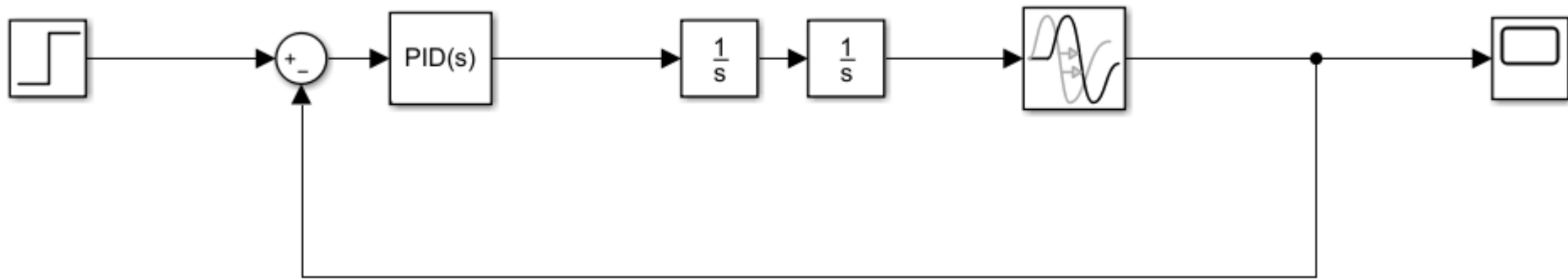
Depends on processing speed

- Queuing

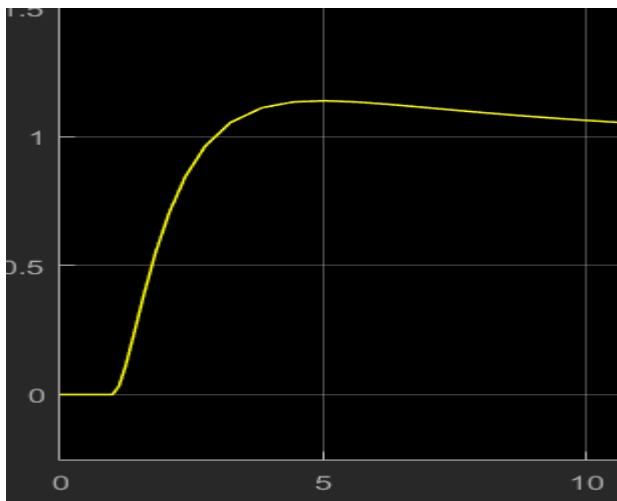
When resources are shared, e.g. serial lines, processing

Consequences of delay

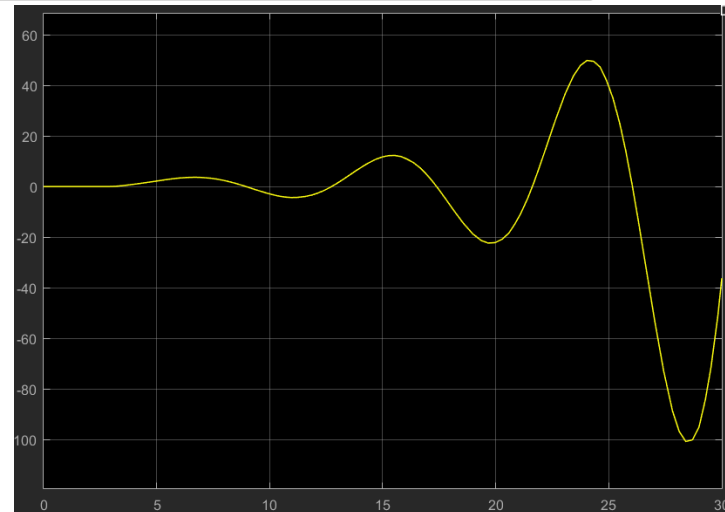
- Control systems



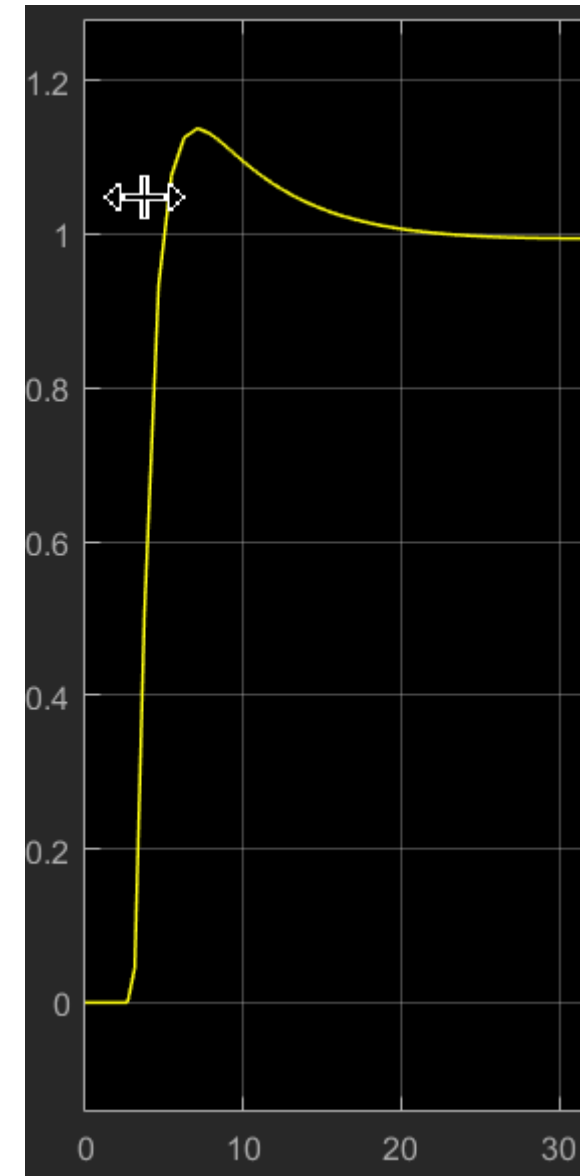
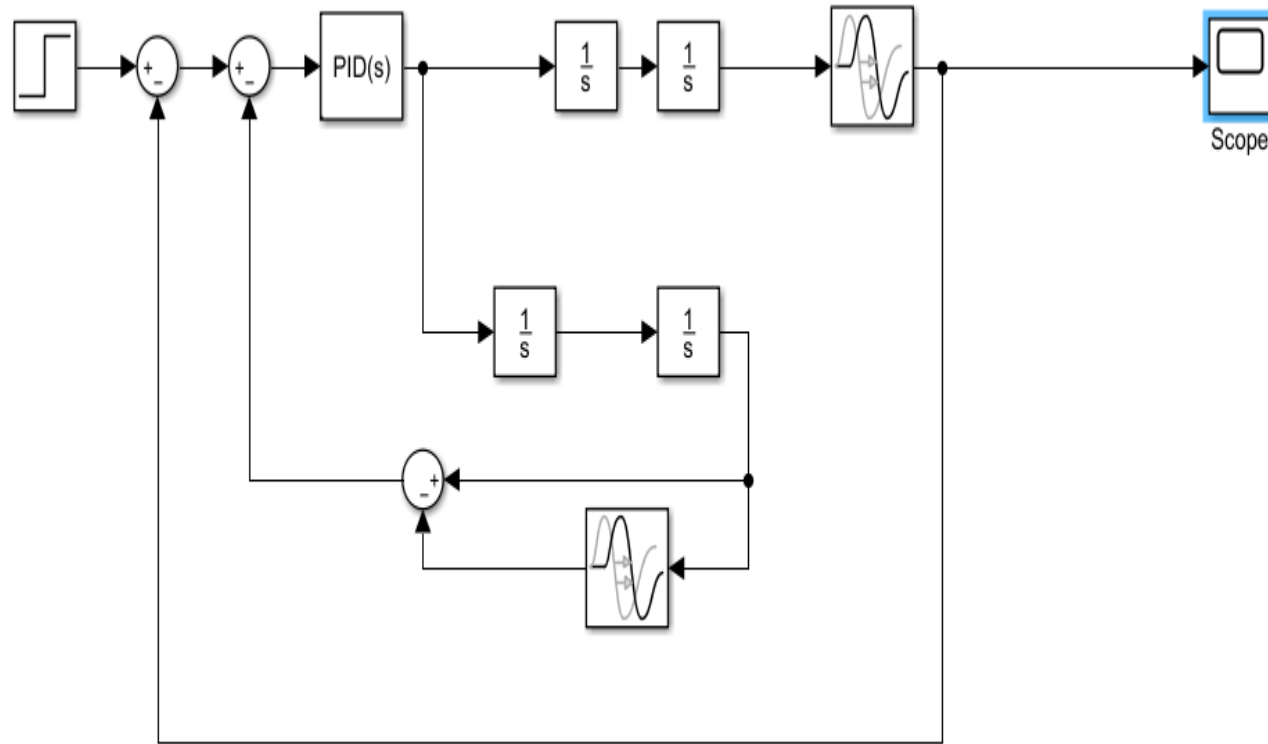
Without
delay



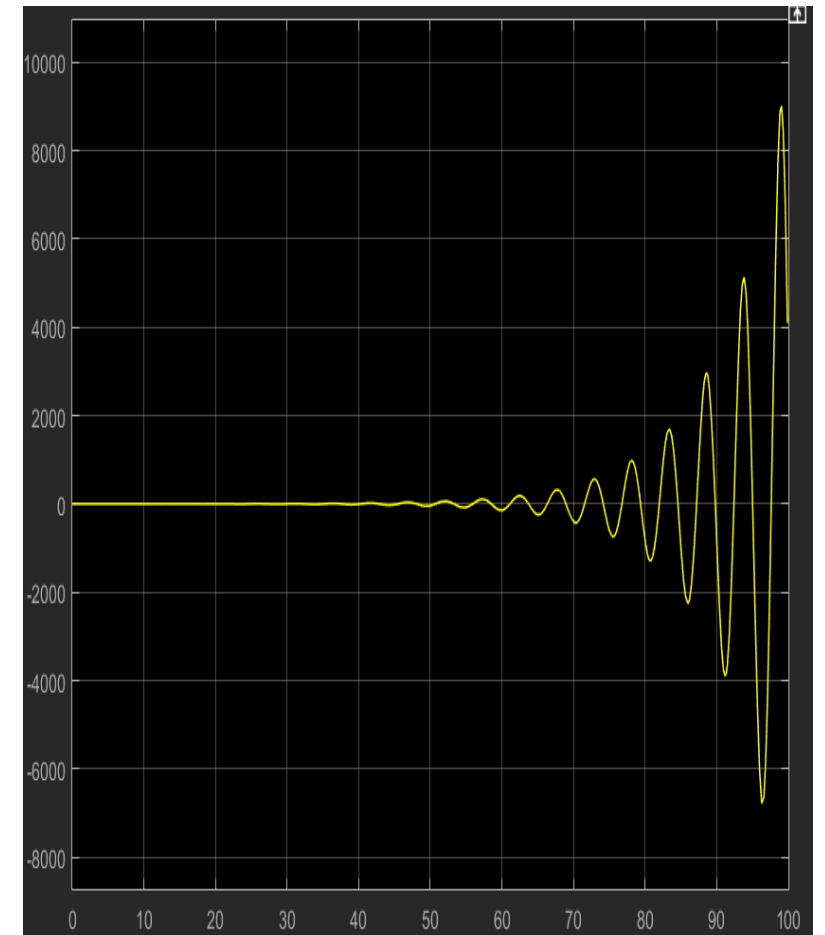
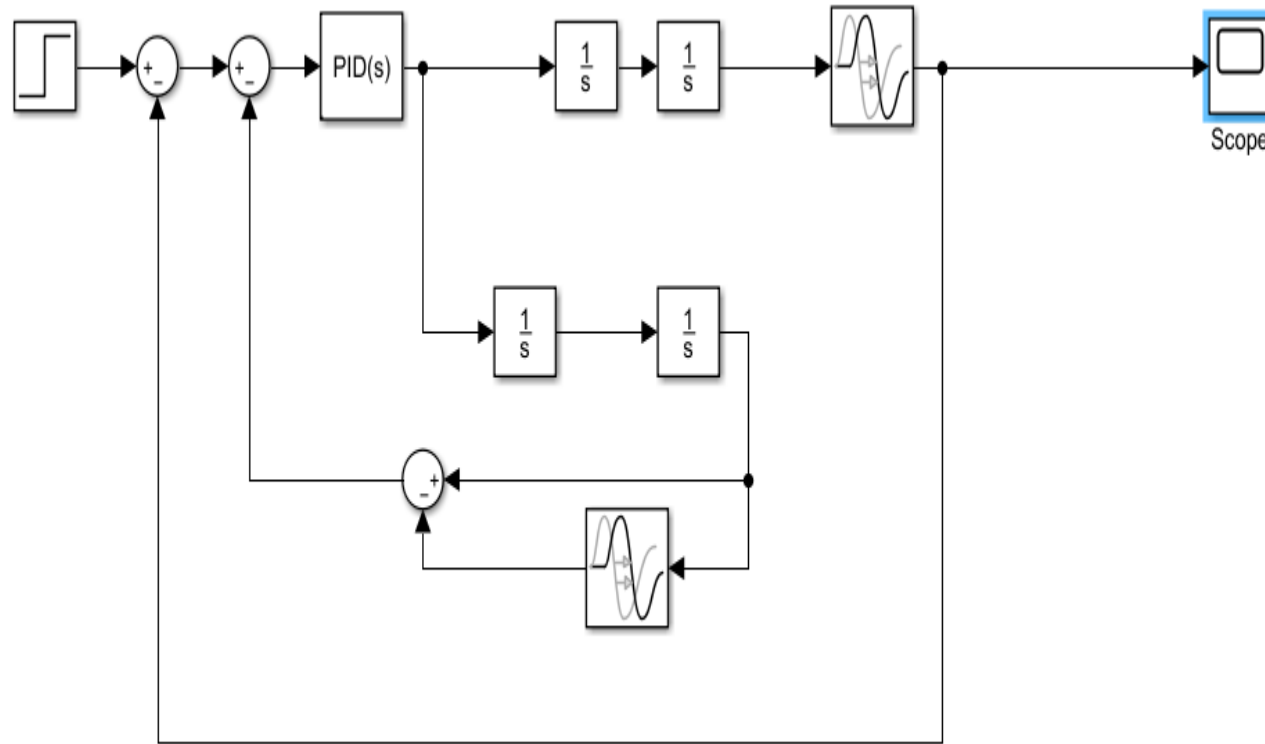
With delay



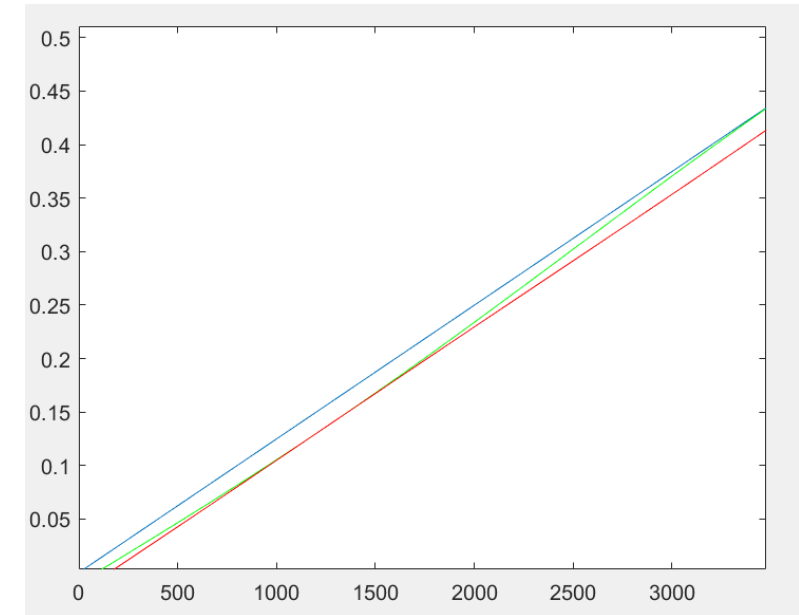
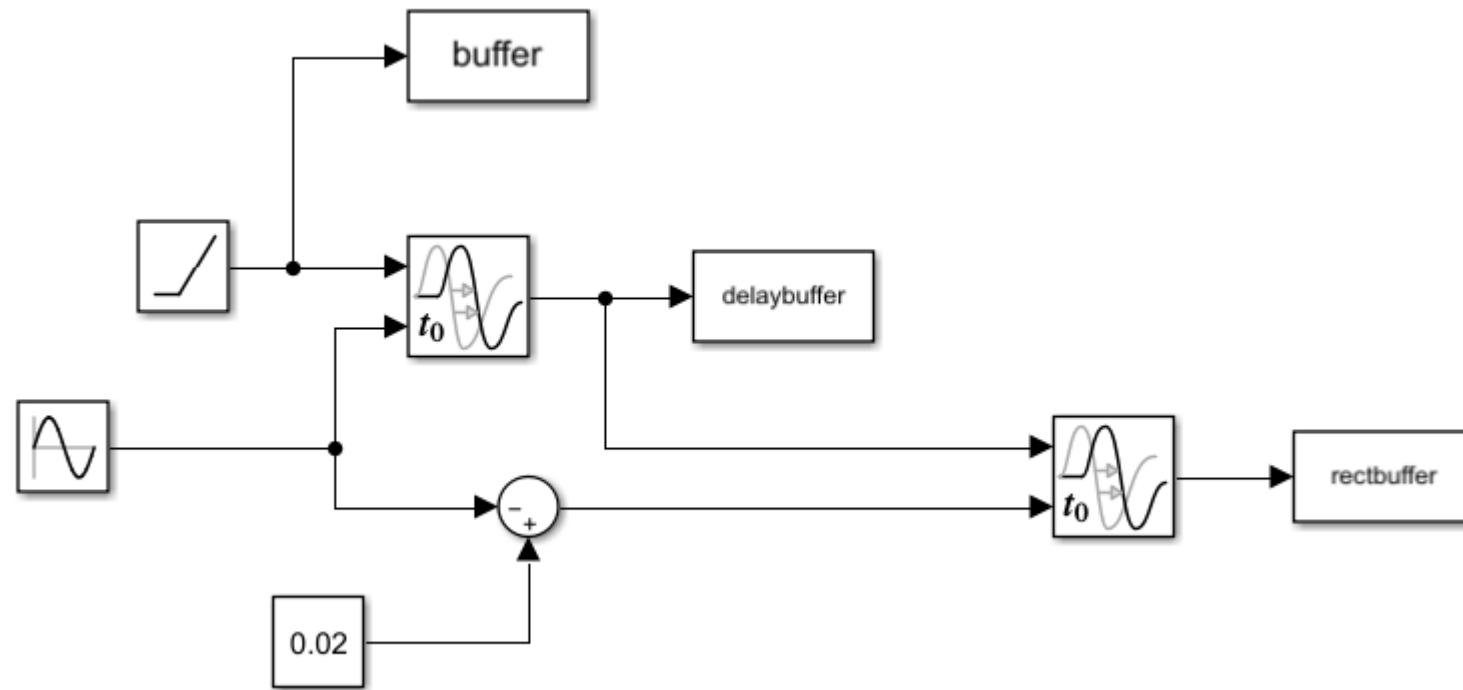
The Smith Predictor



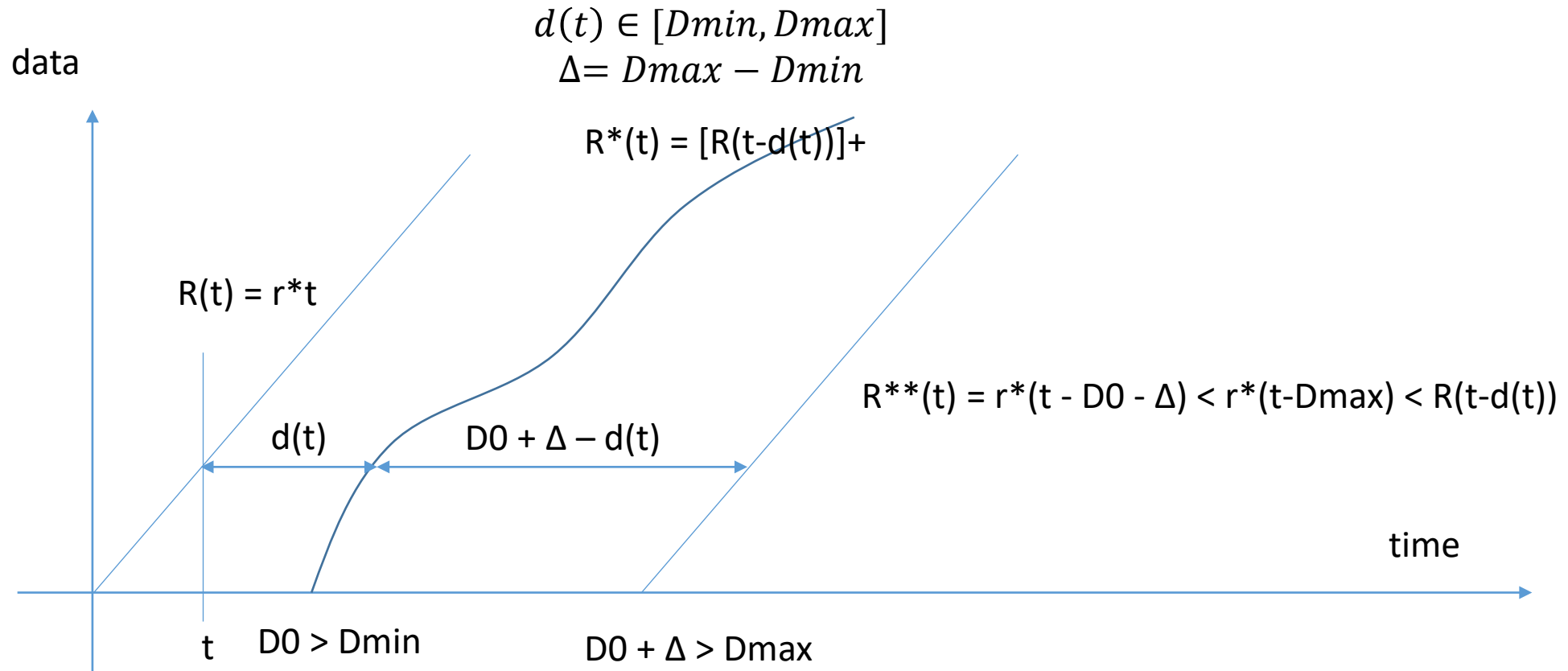
The Smith Predictor with over-estimation



Delay in Communication System (VoIP)



Playout buffer



Approaches to performance assessment

- Analysis
- Simulation
- Testing

Approaches to analysis

- Deterministic
- Non-deterministic
 - Scheduling theory
 - Network/realtime calculus
- Probabilistic/Stochastic
 - Markov chains
 - Queueing Theory

Exercise 1

- $N=8$ sensors each produce periodically 100 bits of measurements data every $T=1\text{mS}$.
- Data are to be sent across a shared communication line offering $s=1\text{Mbps}$ of serialization speed (bandwidth).
- Data from each sensor will be packetized with N (tbd) data points together with an overhead (header, error detection ...) of $O=100$ bits
- Determine the best value for N taking into account delay and bandwidth.
- For arbitrary (variable) N , determine a DNC arrival constraint for each sensor as well as the aggregate arrival curve for all sensors.

Exercise 2

- Encode the arrival constraint of the aggregated flow (from ex 1) for all sensors as an `rtccurve`
- Encode the service curve of the communication line into `rtc`.
- Use `rtch` and `rtcplot` to determine the experienced delay
- Use `rtcv` and `rtcplotv` to determine the experienced backlog
- Use an appropriate tool in the `rtc-toolbox` to determine the optimal value for N

Exercise 3

- Consider 3 nodes on a single CANbus segment with the following periods and priorities:
 - 0: {2mS, 000...}
 - 2: {3mS, 0002...}
 - 1: {5mS, 0001...}
- Assume packet lengths of 100 bits.
- Determine the smallest bitrate on the CANbus for the traffic to be executable.
- Make an RTC-model for the 3 nodes and CANbus with arbitrary bitrate. Include a receive buffer in the lowest priority node as well as a periodic receiver with arbitrary period.
- Determine maximum p2p delay for node 2 as well as minimum receive buffer size (for not loosing packets) for arbitrary receive periods and bitrates.